

Hackathon Final Report - Willow Haughey, 25379396

ABSTRACT

Project Title: We, The Consumer

Team Name: Hackathon Tonight, Queen?

Chosen Theme: Old is Gold

Team Members & Roles:

Misia Dawicka	Willow Haughey
25536189	25379396
Team Lead	Primary Contact
Presentations	GitHub Documentation
Reports	Code Development
Hardware	Architecture
System Testing	

Problem statement & Proposed solution:

With every day that passes, online video streaming becomes more and more of a nightmare with data harvesting, AI content, adverts every five seconds, poor moderation, paywalling, and the algorithm pushing creators to only make content that caters to the majority while ignoring smaller demands for more niche types of content. Sites like YouTube have a monopoly when it comes to consumer-created content and has an easier time staying on top due to it being owned by a bigger and richer corporation like Google.

We will attempt to solve this issue by making a video hosting server which will allow users to upload their own videos. We will host this server using old P.C.s. We will add options for playing audio when the device's display is off, skipping specific chosen sections on videos based off of community timestamps (which can be toggled), optional onscreen comments as well as comments made at specific points, lyrics for music and subtitles for video with changeable fonts to be dyslexia/blindness friendly, and adjustable video quality/bandwidth usage.

METHODOLOGY

- **Functional Requirements:**

Functional requirement 1:

Users shall be able to upload their own files from the frontend. They shall be able to upload a file without having to push it through from the 11ty backend.

Functional requirement 2:

The server shall be hosted on a static website using 11ty to host. This means that most updates to the site will be pushed from the backend, with 11ty as a powerful tool for making and hosting static sites.

Functional requirement 3:

The server shall allow users to retrieve and play media files that have been uploaded. For example, A user shall be able to view any of the videos that are currently uploaded to the server.

Functional requirement 4:

The server shall allow users to create an account using an email and password. It shall also facilitate them logging in using said credentials, even from another device.

Functional requirement 5:

The server shall be able to convert assorted media types to a common filetype of .MP4. This means that regardless of the format uploaded, the site will be able to play the files back from a common format.

Functional Requirement 6:

The server shall be run off a PC using the Ubuntu Server OS.

Functional requirement 7:

Users shall not require extensive knowledge of servers and html to upload, download, login or otherwise interact with the server

- **Non-Functional Requirements:**

Non-Functional requirement 1:

The server shall be able to support 4 concurrent users streaming media without suffering severely performance wise. The server aims to be able to do this without suffering a loss of streaming quality or of site speed.

Non-Functional requirement 2:

The server shall have a UI that is pleasant to look at on various types of devices. The interface shall be easy for users to navigate regardless of their knowledge of media streaming.

Non-Functional requirement 3:

The server shall be able to hold no less than 5 large files of varying length and quality from each of the aforementioned users at the same time without having a heavy impact to the performance. These performance issues would be the same as mentioned above.

Non-Functional requirement 4:

The server shall be able to load a requested video in a timely manner. Users shall be able to download videos locally before watching them.

Non-Functional requirement 5:

The server shall have sufficient bandwidth and storage capacity to be able to handle no less than 2 concurrent uploads at a time to the server.

Non-Functional requirement 6:

The server shall facilitate some sort of bookmark system or similar way for users to “save” media pieces to be found again quicker in the future.

IMPLEMENTATION

-Team Participation and Management:

For this project we assigned each team member a handful of tasks, Willow was assigned to be the primary contact, github documentor and code developer while Misia was assigned to be the team lead, presenter, and system tester. Each member was to complete these tasks both in collaboration with each other and in their own time with their own resources.

-Approach used to manage project while addressing Functional Requirements and Non-Functional Requirements:

Before project development began, concrete goals were set in line with how we wanted the project to operate and look. These goals were then used during the project to compare current stages to goal stages to see what kind of progress was being made.

-How were meetings and project progress managed during the last 2 weeks of the project and whether or not chat software or any professional project management tools were used

No professional software was used for project management outside of Github. All meetings and project discussions were held in person.

-Which Functional Requirements and Non-Functional Requirements were not met:

- **Functional Requirements:**

Functional requirement 1: Incomplete.

Functional requirement 2: Incomplete.

Functional requirement 3: Incomplete.

Functional requirement 4: Incomplete.

Functional requirement 5: Incomplete

Functional Requirement 6: Complete.

Functional requirement 7: Incomplete.

- **Non-Functional Requirements:**

Non-Functional requirement 1: Incomplete.
Non-Functional requirement 2: Incomplete.
Non-Functional requirement 3: Incomplete.
Non-Functional requirement 4: Incomplete.
Non-Functional requirement 5: Incomplete.
Non-Functional requirement 6: Incomplete.

-System Architecture:

Hardware components:

- I5 4300 Thinkpad with 8gb RAM, 224gb SSD, ADAT 6gb/s Sata harddrive, 1gb network card

Software components:

- Ubuntu Server operating system
- 11ty Static server hosting site to host website
- File transfer protocol server on backend
- html/js based frontend - website interface for users to interact with

-Technical implementation decisions:

We decided to use 11ty because it was demonstrated to us by acquaintances its usefulness as a static site generator, we decided to use an Ubuntu server because we figured that it would have a good ratio of simplicity to the amount of features that would be optimal for a project like this, and we chose to use an older Lenovo Thinkpad laptop for our hardware over a desktop P.C. due to its portability and maintainability.

-Challenges encountered and respective solutions:

Half way through developing the functional requirements document, we realised that we didn't know what software we would use for storing files, however after asking around we were recommended 11ty as a backend. Throughout the project we knew there would be a struggle to keep up with deadlines if we were to not use GenAI, and this ended up detracting from the overall final quality of the project. Many of the requirements we had set out to achieve were not met on account of this, as the scope of the project ultimately proved to be far beyond our skill level. Between phase 2 and 3 there was trouble with the provided hardware due to there being no RAM included; the solution we found to this was to use hardware that we already owned that met the requirements given beforehand.